CLAUDE.md — FuelingSense Running Metabolic Calculator

Stewart Sports Ltd · Technical Specification v2.0

Calibrated from Taguchi L8 INSCYD Ground Truth (28.04.2026)

Project Overview

Build a single-page running metabolic calculator that takes **distance + time** from three field tests (20 s sprint, 3-minute run, 6-minute run) plus basic anthropometrics, and outputs a full INSCYD-equivalent metabolic profile: VO2max, VLamax, MLSS, LT1, FATmax, MFO, CARB90,
training zones with substrate g/h values, and an athlete classification.

Output file: `running-metabolic-calculator.html`

Input Specification

All speed values are derived internally. Users enter **distance (m) and time (mm:ss)** only.

```
SPRINT TEST    Distance: ___ m    Time: 0:00:__  (target ~20 s, accept 12–25 s)
3-MIN TEST     Distance: ___ m    Time: 3:00:00  (fixed 180 s)
6-MIN TEST     Distance: ___ m    Time: 6:00:00  (fixed 360 s)

ANTHROPOMETRICS
  Body mass (kg)        range: 40–130
  Body fat %            range: 5–35
  Fat-free mass (kg)    = mass × (1 − bf/100)  [auto-calculated, display only]
```

Speed derivation:

```
const S20  = distSprint / timeSprint;   // m/s
const S180 = dist3min   / 180;          // m/s
const S360 = dist6min   / 360;          // m/s
```

Input validation (hard blocks — prevent calculation):

```
if (S20 <= S180) error("Sprint must be faster than 3-min pace");
if (S180 <= S360) error("3-min must be faster than 6-min pace");
```

```
if (S20 < 3.0 || S20 > 12.0) error("Sprint speed out of range");
if (S360 < 1.0 || S360 > 7.5) error("6-min speed out of range");
```

Live display (update as user types):

- Show pace (min:sec/km) alongside each entered effort
- Show fat-free mass auto-calculated

Stage 1 — Estimate VO2max and VLamax from Speed Tests

These are the primary regression equations fitted to the 19-profile synthetic dataset and validated against 8 Taguchi L8 INSCYD profiles.

VO2max

```
// Primary: 6-min speed (R² = 0.9892)
const VO2max = 11.5472 * S360 + 5.5136;   // mL/kg/min

// Clamp to physiological range
const VO2max_clamped = Math.max(20, Math.min(85, VO2max));
```

VLamax

```
// Three-speed model (R² = 0.9129)
// sprint_decay captures glycolytic contribution relative to aerobic capacity
const sprint_decay = S20 - S360;

const VLamax = 0.0708 * S20
             + 0.2785 * S180
             - 0.3400 * S360
             - 0.0745;

// Clamp to physiological range
const VLamax_clamped = Math.max(0.05, Math.min(0.80, VLamax));
```

Note on sprint_decay coefficient sign: The negative S360 coefficient is physiologically correct — a higher 6-min speed at the same sprint speed implies greater aerobic contribution to the sprint, meaning LOWER glycolytic power. Do not change the sign.

MLSS (Anaerobic Threshold / MaxLass Steady State)

```
// Direct regression from speed tests (R² = 0.9963)
const MLSS = 0.1052 * VO2max_clamped
           - 1.4138 * VLamax_clamped
           - 1.8479;

const MLSS_clamped = Math.max(1.0, Math.min(7.0, MLSS));
```

Stage 2 — Substrate Cascade (v2.0, Taguchi L8 Calibration)

CRITICAL DIFFERENCE FROM BIKE APP:
In cycling, power normalises economy so VLamax alone predicts FATmax position.
In running, absolute speed does NOT normalise economy — VO2max must appear as an explicit term. The bike formula (xf = 0.700 + 0.080 × ln(0.55/VLa)) overpredicts running xf by 18% on average. Use the running-specific formulae below exclusively.

2a. LT1 Speed

```
const ln_vla = Math.log(0.55 / VLamax_clamped);

const lt1_frac = 0.4273
               + 0.0982 * ln_vla
               + 0.2241 * (VO2max_clamped / 50);

const LT1_frac = Math.max(0.35, Math.min(0.90, lt1_frac));
const LT1 = LT1_frac * MLSS_clamped;    // m/s
```

2b. FATmax Position and Speed

```
// xf = fraction of MLSS at which fat oxidation peaks
const xf_raw = 0.1124
             + 0.1131 * ln_vla
             + 0.4073 * (VO2max_clamped / 50);

const xf = Math.max(0.25, Math.min(0.80, xf_raw));
const FATmax_speed = xf * MLSS_clamped;   // m/s
```

Validation: $R^2 = 0.9689$, RMSE = 0.022 (n=8 INSCYD profiles)
Calibration range: VLamax 0.10–0.54, VO2max 34–62, MLSS 1.39–4.97 m/s

2c. MFO — Maximum Fat Oxidation (Running G6 Formula)

```
const RE_CONST  = 12.22;   // mL O2 / kg / km  (running economy constant)
const O2_KCAL   = 5.0;     // kcal / L O2
const FAT_KCAL  = 9.3;     // kcal / g fat
const CHO_KCAL  = 4.1;     // kcal / g carbohydrate

// Convert MLSS speed to energy rate
const MLSS_kcal_h = MLSS_clamped * RE_CONST * (60 / 1000) * O2_KCAL * mass;

// G6 base formula
const fatmaxBase = Math.max(0,
    0.057224 * MLSS_kcal_h - (0.3842 * VLamax_clamped - 0.0149) * mass
);

// VLamax scaling factor
const vla_norm = Math.max(-1, Math.min(1, (0.55 - VLamax_clamped) / 0.25));
const fatScale  = 1 + 0.06 * vla_norm;
```

```
const MFO_g_h    = fatmaxBase * fatScale;
const MFO_kcal_h = MFO_g_h * FAT_KCAL;
```

Validation: $R^2 = 0.9659$, RMSE = 36 kcal/h (n=8 INSCYD profiles)

2d. CHO Fraction Sigmoid (Full Substrate Curve)

```
// Logistic function
function sigmoid(x) { return 1 / (1 + Math.exp(-x)); }

// CHO fraction at normalised speed v_n = v / MLSS
function fCHO(v_n, vlamax) {
    return sigmoid(
        -5.3519
        + 5.6460 * v_n
        + 5.9029 * vlamax
        - 3.7934 * v_n * vlamax
    );
}

// Substrate rates at any speed v (m/s)
function substrateAtSpeed(v, vo2max, vlamax, mlss, mass) {
    const v_n  = v / mlss;
    const E_dot = v * RE_CONST * (60 / 1000) * O2_KCAL * mass;  // kcal/h total
    const f_cho = fCHO(v_n, vlamax);
    return {
        cho_g_h: (f_cho * E_dot) / CHO_KCAL,
        fat_g_h: ((1 - f_cho) * E_dot) / FAT_KCAL,
        total_kcal_h: E_dot
    };
}
```

2e. CARB90 Detection

CARB90 = the speed at which CHO oxidation first crosses 90 g/h (adapted from bike A3).

```
function detectCARB90(vo2max, vlamax, mlss, mass) {
    const v_max  = mlss * 1.15;
    const step   = 0.01;
    const speeds = [];
    for (let v = 0.5; v <= v_max; v += step) speeds.push(v);

    // Compute CHO at each speed
    let cho_series = speeds.map(v => substrateAtSpeed(v, vo2max, vlamax, mlss, mass)

    // Apply monotonic constraint (from bike A3)
    for (let i = 1; i < cho_series.length; i++) {
        if (cho_series[i] < cho_series[i - 1]) cho_series[i] = cho_series[i - 1];
    }

    // Find first crossing of 90 g/h
    for (let i = 0; i < cho_series.length - 1; i++) {
```

```
        if (cho_series[i] < 90 && cho_series[i + 1] >= 90) {
            // Linear interpolation
            const carb90 = speeds[i]
                + (90 - cho_series[i])
                * (speeds[i + 1] - speeds[i])
                / (cho_series[i + 1] - cho_series[i]);
            return carb90;
        }
    }
    // Never reached — return max scan speed
    return v_max;
}
```

Validation: RMSE = 0.211 m/s, Bias = +0.033 m/s (n=7 profiles reaching 90 g/h)

Stage 3 — Derived Metrics

```
// Fat-free mass
const FFM = mass * (1 - bf / 100);

// Relative VO2max (already relative — input is mL/kg/min)
const VO2max_rel = VO2max_clamped;
const VO2max_abs = VO2max_clamped * mass;   // mL/min

// %VO2max at MLSS
const pctVO2atMLSS = (MLSS_clamped / S360) * 100;   // approx

// Glycogen estimate: 15 g/kg FFM for base, scaled by VLamax
const glycogen_g_kg = Math.min(9, Math.max(3, 4.5 + 2.0 * (1 - VLamax_clamped / 0.55)
const glycogen_abs  = glycogen_g_kg * FFM;

// Aerobic-glycolytic ratio (qualitative index)
const AG_ratio = VO2max_clamped / (VLamax_clamped * 100 + 1);

// vVO2max (speed at which VO2max is reached) — approx = 1.05–1.10 × MLSS
const vVO2max = MLSS_clamped * (1 + 0.07 * (1 - VLamax_clamped / 0.55));
```

Stage 4 — Training Zones

Generate 8 zones from LT1 and MLSS. Compute substrate g/h per zone using `substrateAtSpeed`.

```
function buildZones(lt1, mlss, vo2max, vlamax, mass) {
    const vVO2 = mlss * 1.08;

    const zones = [
        { name: 'Recovery',          code: 'REC',  lo: 0.60, hi: 0.75, ref: lt1  },
        { name: 'Base / Aerobic',    code: 'BASE', lo: 0.75, hi: 0.90, ref: lt1  },
        { name: 'Tempo',             code: 'TMP',  lo: 0.90, hi: 1.00, ref: lt1  },
        { name: 'FATmax',            code: 'FAT',  lo: 0.85, hi: 0.97, ref: 'fatmax'
        { name: 'Threshold (MLSS)',  code: 'AT',   lo: 0.97, hi: 1.03, ref: mlss },
```

```
        { name: 'Aerobic Max',        code: 'AMAX', lo: 1.03, hi: 1.10, ref: mlss },
        { name: 'High Anaerobic',     code: 'ANAX', lo: 1.10, hi: 1.25, ref: mlss },
        { name: 'Lactate Shuttle',    code: 'LAEX', lo: 0.80, hi: 1.08, ref: 'range'
    ];

    return zones.map(z => {
        let v_lo, v_hi, v_target;
        if (z.ref === 'fatmax') {
            v_target = xf * mlss;
            v_lo     = v_target * 0.95;
            v_hi     = v_target * 1.05;
        } else if (z.ref === 'range') {
            v_lo     = lt1 * 0.90;
            v_hi     = mlss * 1.05;
            v_target = (v_lo + v_hi) / 2;
        } else if (z.ref === lt1) {
            v_target = lt1 * ((z.lo + z.hi) / 2);
            v_lo     = lt1 * z.lo;
            v_hi     = lt1 * z.hi;
        } else {
            v_target = mlss * ((z.lo + z.hi) / 2);
            v_lo     = mlss * z.lo;
            v_hi     = mlss * z.hi;
        }
        const sub    = substrateAtSpeed(Math.max(0.5, v_target), vo2max, vlamax, mls
        const pace   = v => `${Math.floor(1000/v/60)}:${String(Math.round((1000/v/60 %
        return {
            ...z,
            v_lo, v_hi, v_target,
            pace_lo: pace(v_hi),
            pace_hi: pace(v_lo),
            pace_target: pace(v_target),
            fat_g_h:   Math.round(sub.fat_g_h),
            cho_g_h:   Math.round(sub.cho_g_h),
            kcal_h:    Math.round(sub.total_kcal_h),
            pct_fat:   Math.round((sub.fat_g_h * FAT_KCAL / sub.total_kcal_h) * 100),
            pct_cho:   Math.round((sub.cho_g_h * CHO_KCAL / sub.total_kcal_h) * 100),
        };
    });
}
```

Stage 5 — Guard Conditions and Confidence Flags

```
function assessConfidence(vo2max, vlamax, mlss, s20, s180, s360) {
    const flags = [];

    // Extreme corner: high glycolytic + low aerobic
    if (vlamax > 0.50 && vo2max < 38) {
        flags.push({
            level: 'warn',
            msg: 'High VLamax + low VO₂max profile: MFO and CARB90 estimates have re
        });
    }
```

```
    // Sprint speed hierarchy violated (shouldn't reach here if input validation is
    if (s20 <= s180) flags.push({ level: 'error', msg: 'Sprint must be faster than 3-
    if (s180 <= s360) flags.push({ level: 'error', msg: '3-min must be faster than 6-

    // Very low VLamax — endurance specialist
    if (vlamax < 0.08) {
        flags.push({ level: 'info', msg: 'Extremely low VLamax: elite endurance prof
    }

    // MLSS > VO2max — physiologically impossible
    const vo2_at_mlss = mlss * RE_CONST;   // mL/kg/min approx
    if (vo2_at_mlss > vo2max * 1.02) {
        flags.push({ level: 'error', msg: 'MLSS speed implies O₂ demand exceeds VO₂
    }

    return flags;
}
```

UI Architecture

Layout

Two-column dashboard on desktop (≥768px), single-column on mobile.

```
┌──────────────────────────────────────────────────────────────┐
│  Header: SVG logo + "Running Metabolic Profile" + theme toggle│
├──────────────────┬───────────────────────────────────────────┤
│  LEFT PANEL      │  RIGHT PANEL                              │
│                  │                                           │
│  Input Form      │  Results Dashboard                        │
│  ──────────      │  ─────────────────                        │
│  Sprint effort   │  4 × Metric Cards (primary outputs)       │
│  3-min effort    │  Metabolic Spectrum Bar                   │
│  6-min effort    │  Substrate Curve Chart (fat+CHO vs pace)  │
│  Body mass       │  Training Zones Table                     │
│  Body fat %      │  Athlete Classification Card              │
│  ──────────      │  Confidence flags (if any)                │
│  [Calculate]     │                                           │
└──────────────────┴───────────────────────────────────────────┘
```

4 Primary Metric Cards

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│  VO₂max      │  │  VLamax      │  │  MLSS        │  │  MFO         │
│  XX.X        │  │  X.XX        │  │  X:XX/km     │  │  XXX kcal/h  │
│  mL/kg/min   │  │  mmol/L/s    │  │  (X.XX m/s)  │  │  XX g/h fat  │
│  ████▒▒▒▒    │  │  ████▒▒▒▒    │  │  ████▒▒▒▒    │  │  ████▒▒▒▒    │
│  gauge bar   │  │  gauge bar   │  │  gauge bar   │  │  gauge bar   │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘
```

Gauge bar reference ranges:

- VO2max: 30 (poor) → 80 (elite) mL/kg/min
- VLamax: 0.05 (pure endurance) → 0.70 (power/sprint)
- MLSS: 2.0 → 6.0 m/s
- MFO: 100 → 800 kcal/h

Metabolic Spectrum Bar

A horizontal bar from LT1 to vVO2max with markers at:

- LT1 (green label)
- FATmax (gold label)
- MLSS (orange label, thicker marker)
- CARB90 (red label)

All displayed as pace (min:sec/km) beneath the marker.

Substrate Curve Chart

Use Chart.js. X-axis: pace (min:sec/km) from easy to fast.
Two filled area lines: fat g/h (green fill) and CHO g/h (orange fill).
Vertical marker lines at LT1, FATmax, MLSS, CARB90.
Show CHO = 90 g/h as dashed horizontal reference.
Update in real-time with every calculation.

```
// Generate curve data
function buildSubstrateCurve(vo2max, vlamax, mlss, mass) {
    const speeds = [];
    for (let v = 0.8; v <= mlss * 1.15; v += 0.05) speeds.push(v);
    return speeds.map(v => {
        const s = substrateAtSpeed(v, vo2max, vlamax, mlss, mass);
        const pace_sec = 1000 / v;
        const pace_str = `${Math.floor(pace_sec/60)}:${String(Math.round(pace_sec%60)
        return { pace: pace_str, fat: s.fat_g_h, cho: s.cho_g_h, speed: v };
    });
}
```

Training Zones Table

Columns: Zone | Code | Pace Range | Kcal/h | %Fat | %CHO | Fat g/h | CHO g/h

Use `font-variant-numeric: tabular-nums` on all numeric cells.
Highlight threshold row (AT) with accent background.
Highlight FATmax row with gold background.

Athlete Classification

Four-quadrant classification based on VO2max and VLamax:

```
function classifyAthlete(vo2max, vlamax) {
    const highVO2  = vo2max  >= 50;
    const highVLa  = vlamax  >= 0.30;

    if ( highVO2 &&  highVLa) return { type: 'Power-Endurance',  desc: 'Strong aerob
    if ( highVO2 && !highVLa) return { type: 'Pure Endurance',   desc: 'Elite endurar
    if (!highVO2 &&  highVLa) return { type: 'Power-Glycolytic', desc: 'Glycolytic st
    if (!highVO2 && !highVLa) return { type: 'Base Builder',     desc: 'Moderate aer
}
```

Design System

Colours (4-metric colour coding)

```
:root {
    --color-vo2:   #01696f;   /* teal    — aerobic */
    --color-vla:   #da7101;   /* orange  — glycolytic */
    --color-mlss:  #437a22;   /* green   — threshold */
    --color-mfo:   #d19900;   /* gold    — fat */
    --color-carb:  #a12c7b;   /* magenta — carbohydrate */
    --color-lt1:   #006494;   /* blue    — aerobic base */
}
```

Typography

```
<link href="https://api.fontshare.com/v2/css?f[]=space-grotesk@400,500,700&f[]=cabine
```

- Display/headings: Space Grotesk
- Body/numbers: Cabinet Grotesk
- All pace/speed values: `font-variant-numeric: tabular-nums`

Pace Formatting

```
function formatPace(speed_ms) {
    if (!speed_ms || speed_ms <= 0) return '—';
    const sec_per_km = 1000 / speed_ms;
    const min = Math.floor(sec_per_km / 60);
    const sec = Math.round(sec_per_km % 60);
    return `${min}:${String(sec).padStart(2, '0')}/km`;
}
```

Equation Summary (Quick Reference for Implementation)

```
// ── STAGE 1: Speed → VO2max, VLamax, MLSS ──────────────────
const S20  = dSprint / tSprint;
const S180 = d3min   / 180;
const S360 = d6min   / 360;

const VO2max  = clamp(11.5472 * S360 + 5.5136, 20, 85);
const VLamax  = clamp(0.0708*S20 + 0.2785*S180 - 0.3400*S360 - 0.0745, 0.05, 0.80);
const MLSS    = clamp(0.1052*VO2max - 1.4138*VLamax - 1.8479, 1.0, 7.0);

// ── STAGE 2: Substrate cascade (v2.0 — Taguchi L8 calibrated) ─
const ln_vla  = Math.log(0.55 / VLamax);

// LT1
const LT1_frac = clamp(0.4273 + 0.0982*ln_vla + 0.2241*(VO2max/50), 0.35, 0.90);
const LT1      = LT1_frac * MLSS;

// FATmax position (RUNNING-SPECIFIC — NOT the bike formula)
const xf       = clamp(0.1124 + 0.1131*ln_vla + 0.4073*(VO2max/50), 0.25, 0.80);
const FATmax_v = xf * MLSS;

// MFO — Running G6
const MLSS_kcal_h = MLSS * 12.22 * (60/1000) * 5.0 * mass;
const base        = Math.max(0, 0.057224*MLSS_kcal_h - (0.3842*VLamax - 0.0149)*mass);
const scale       = 1 + 0.06 * clamp((0.55-VLamax)/0.25, -1, 1);
const MFO_kcal_h  = base * scale * 9.3;

// f_CHO sigmoid
// f_CHO(v_n, vlamax) = σ(-5.3519 + 5.6460*v_n + 5.9029*vlamax - 3.7934*v_n*vlamax)

// CARB90: scan 0.5→1.15×MLSS in 0.01 m/s steps, monotonic CHO, interpolate @90 g/h
```

Calibration provenance:

- Stage 1 equations: 19-profile synthetic dataset + LOO-CV $R^2 > 0.96$
- Stage 2 equations: 8 Taguchi L8 INSCYD running profiles (28.04.2026)
- xf RMSE = 0.022 vs INSCYD | MFO RMSE = 36 kcal/h | CARB90 RMSE = 0.211 m/s

Implementation Sanity Checks (Formula-Derived)

Enter these as test cases. Expected outputs are calculated directly from the v2.0 equations above and should match after rounding. Invalid legacy cases that violate the speed hierarchy checks have been removed.

Input speeds (m/s)	Mass	BF%	Expected VO2	Expected VLa	Expected MLSS	Expected MFO
S20=5.5 S180=3.50 S360=2.83	65	10	~38.2	~0.33	~1.71	~157 kcal/h

Input speeds (m/s)	Mass	BF%	Expected VO2	Expected VLa	Expected MLSS	Expected MFO
S20=8.1 S180=5.50 S360=4.83	65	18	~61.3	~0.39	~4.05	~449 kcal/h

Model Limitations (Display in UI)

Show a collapsible "About this model" section with:

- Equations calibrated on n=8 INSCYD virtual profiles (Taguchi L8 design, 28.04.2026)
- Body composition varies 65–85 kg, 10–18% BF; height fixed at 180 cm
- Running economy assumed constant at 12.22 mL O_2/kg/km
- Low confidence when VLamax > 0.50 AND VO_2max < 38 simultaneously
- Not a substitute for a laboratory INSCYD or VO_2max test
- Next calibration step: L16 array adding height + measured RE variation

Spec version: 2.0 | Calibration date: 28.04.2026 | Stewart Sports Ltd